FaroNet versión 1: Capa Física por Luz (Faro) y Red Mesh Anónima con Enrutamiento por Capas

RFC-FN: 1 · Categoría: Experimental · Noviembre 2026
J. Solís Fuentes · J. A. Solís Mora · Tecnológico de Costa Rica (ITCR)

Resumen

Este documento especifica FaroNet versión 1, una red de mensajería que protege tanto el contenido de los mensajes como sus metadatos (quién habla con quién, cuándo y desde dónde). Define cuatro protocolos: (1) el protocolo de Capa 1 del "Faro", un dispositivo que transmite tramas por luz infrarroja entre dos computadoras; (2) la red mesh FaroNet, que transporta paquetes de tamaño fijo sobre TCP/IP y sobre el Faro, con enrutamiento dinámico por vector de distancias y un directorio de nodos firmado; (3) el enrutamiento por capas ("cebolla") de tres saltos, que impide que un nodo intermedio conozca a la vez el origen y el destino de un mensaje; y (4) la interfaz entre el gateway de FaroNet y una aplicación pública desplegada en el edge (Cloudflare Workers). Incluye las Consideraciones de Privacidad y de Seguridad del diseño.

Tabla de contenidos

- 1. Introducción
- 2. Arquitectura
- 3. Capa 1: Protocolo del Faro
- 4. El Faro como Enlace de la Mesh
- 5. FaroNet: Red Mesh
- 6. Enrutamiento por Capas (Cebolla)
- 7. Gateway y Aplicación Pública en el Edge
- 8. Manejo de Versiones
- 9. Consideraciones de IANA
- 10. Consideraciones de Privacidad
- 11. Consideraciones de Seguridad
- 12. Referencias
- Apéndice A. Vectores de Prueba
- Apéndice B. Mediciones de la Implementación de Referencia
- Apéndice C. Correspondencia con la Implementación
- Direcciones de los Autores

1. Introducción

FaroNet es una red de mensajería construida "desde el cobre hacia arriba". Cada grupo construye un dispositivo de transmisión por luz, el Faro, que une dos comunidades separadas sin otra conexión. Sobre el Faro y sobre redes TCP/IP comunes (Ethernet y WiFi 802.11) corre una red mesh cuyo objetivo principal es transportar mensajes sin revelar quien los originó. Un gateway conecta la mesh con Internet ("clearnet") a través de funciones serverless en el edge, diseñadas para aprender lo mínimo posible de sus usuarios.

Que un mensaje vaya cifrado no basta: sus metadatos permiten vigilar a una persona. Por eso FaroNet:

- usa paquetes de tamaño fijo con relleno aleatorio, para no filtrar la longitud del mensaje;
- no incluye el origen en los paquetes de datos;
- envuelve los mensajes privados en tres capas de cifrado, una por relay, de modo que ningún relay conoce a la vez origen y destino;
- permite rotar la dirección física del Faro y las llaves de cifrado de cada nodo;
- no guarda registros con direcciones IP, direcciones físicas ni identificadores de usuario en ningún componente.

1.1. Alcance

Este documento especifica:

- el protocolo de Capa 1 del Faro: medio, codificación de línea, formato de trama, verificación de integridad, acceso al medio, fragmentación, dirección física y manejo de versiones (Sección 3), y su uso como enlace de la mesh (Sección 4);
- el protocolo de la mesh FaroNet: paquete, saludo, enrutamiento, identidades y directorio (Sección 5);
- el enrutamiento por capas (Sección 6);
- la interfaz del gateway con el edge y los recursos HTTP públicos (Sección 7).

La aplicación de chat y las páginas web del edge solo se describen en lo necesario para la interoperabilidad.

1.2. Convenciones y Terminología

Las palabras clave "DEBE", "NO DEBE", "REQUERIDO", "DEBERÍA", "NO DEBERÍA", "RECOMENDADO", "PUEDE" y "OPCIONAL" en este documento se interpretan como "MUST", "MUST NOT", "REQUIRED", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY" y "OPTIONAL" de BCP 14 [RFC2119] [RFC8174] cuando, y solo cuando, aparecen en mayúsculas.

Todos los enteros de varios bytes se transmiten en orden de red (big-endian). "||" denota concatenación. "0^n" denota n bytes con valor cero. Los tamaños se expresan en bytes salvo que se diga otra cosa.

- **Faro:** dispositivo de transmisión por luz conectado a un nodo (en la implementación de referencia, una Raspberry Pi por su UART).
- **Trama:** unidad de datos de la Capa 1 (como máximo 128 bytes).
- **Dirección física:** identificador de 16 bits de un Faro, análogo a una dirección MAC, rotable.
- **Nodo:** proceso que participa en la mesh FaroNet; se identifica por un nombre de 1 a 16 bytes en UTF-8, sin espacios.
- **Paquete:** unidad de datos de la mesh; mide siempre 512 bytes.
- **Vecino:** nodo con el que existe un enlace directo (conexión TCP o un par de Faros).
- **Ficha:** registro firmado con el nombre de un nodo y sus llaves públicas, publicado en el directorio.
- **Relay:** nodo intermedio que quita una capa de cifrado de una celda.
- **Celda:** cuerpo cifrado por capas que viaja entre relays.
- **Sellado:** mensaje cifrado extremo a extremo hacia el destinatario.
- **Gateway:** nodo de la mesh que publica en el edge los mensajes marcados como públicos.
- **Edge:** aplicación serverless (Cloudflare Workers) que expone salas, feeds, buzón muerto y una réplica del directorio.
- **Maestro / esclavo:** papeles de un Faro al transmitir una trama que requiere confirmación (maestro) o al recibirla y confirmarla (esclavo).

2. Arquitectura

2.1. Componentes

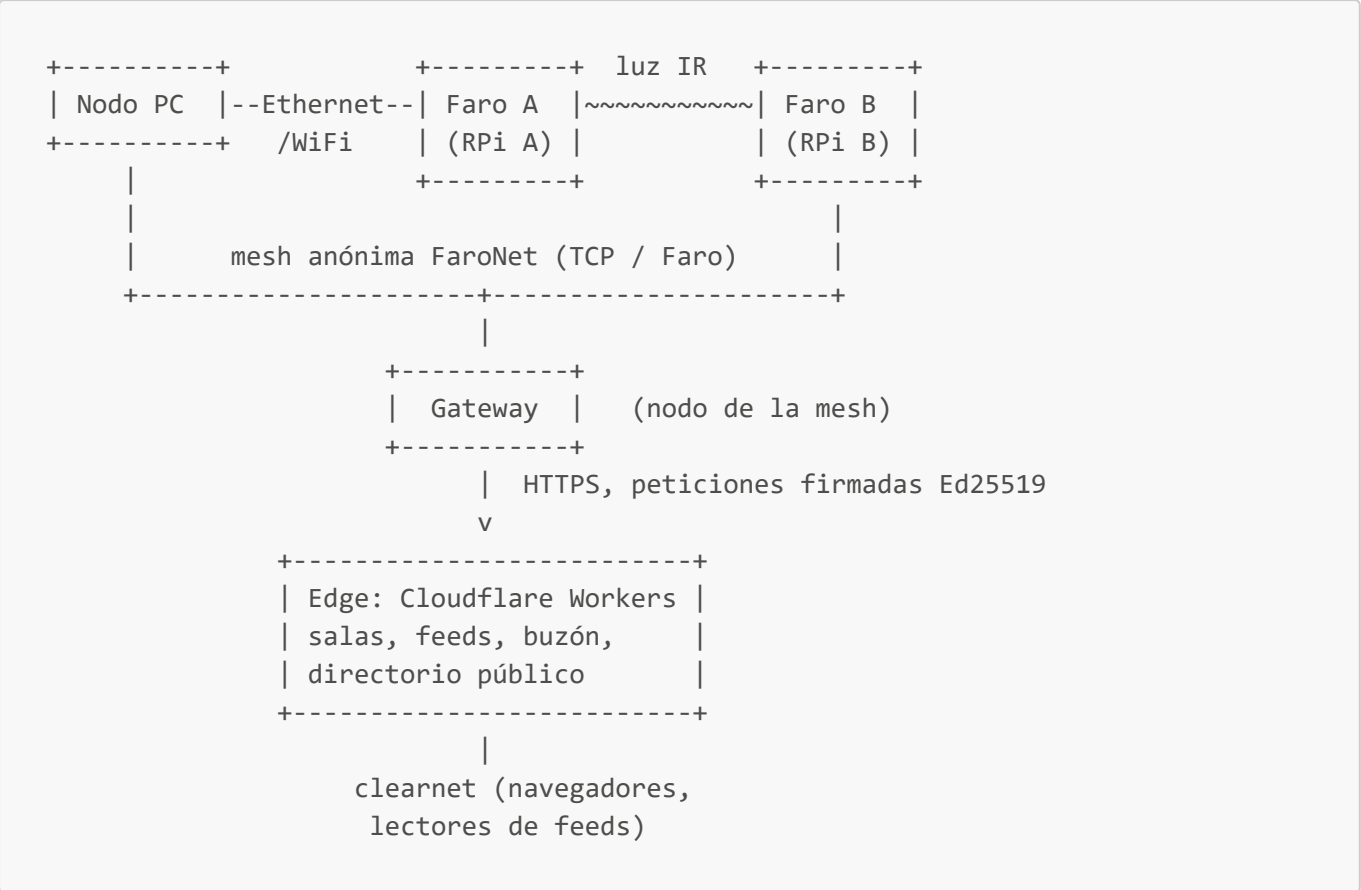


Figura 1: Arquitectura de FaroNet

2.2. Pila de protocolos

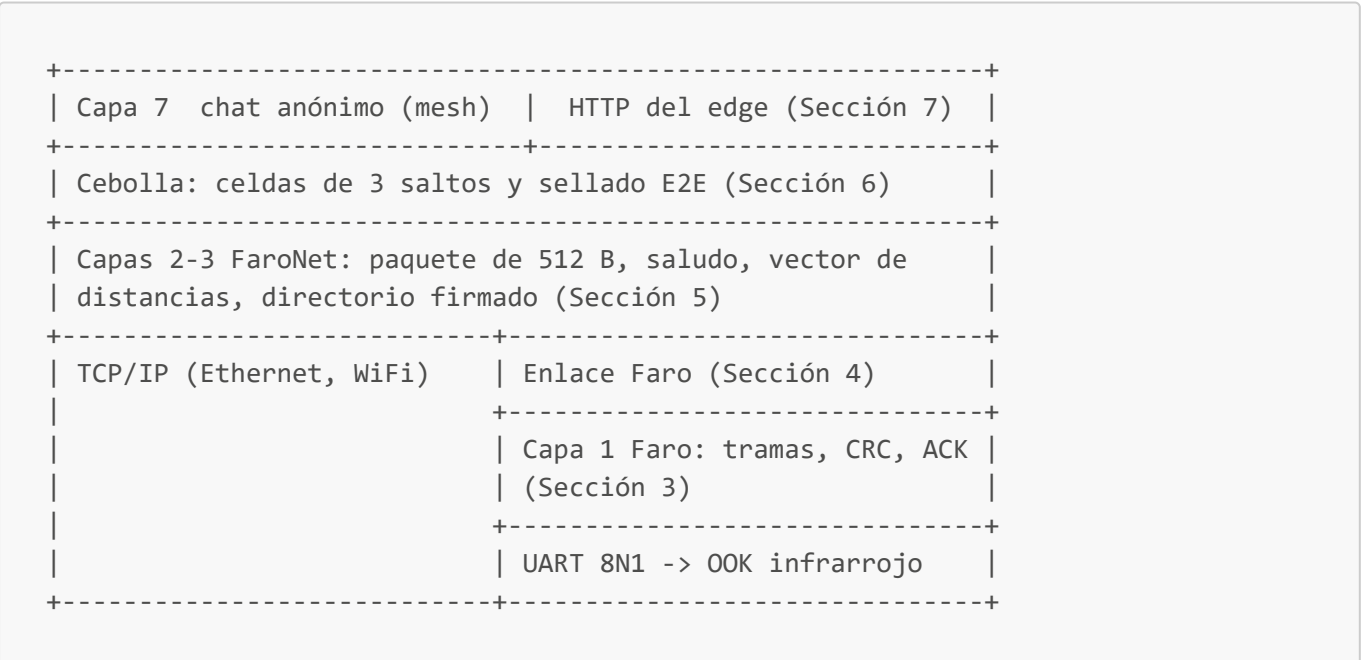


Figura 2: Pila de protocolos

3. Capa 1: Protocolo del Faro

3.1. Medio Físico y Codificación de Línea

El medio es luz infrarroja en espacio libre, con línea de vista entre dos Faros. Cada Faro tiene:

- un emisor: LED infrarrojo con resistencia limitadora, conmutado por un transistor NPN cuya base se conecta a la salida TXD del UART del nodo;
- un receptor: fototransistor con resistencia de carga y un comparador (LM393) con umbral ajustable por potenciómetro, cuya salida (con resistencia de pull-up a 3,3 V) va a la entrada RXD;
- dos LEDs indicadores (TX y RX) y un botón de pánico.

La codificación de línea es OOK (On-Off Keying) con NRZ, generada directamente por el UART en formato 8N1: un bit de inicio (luz apagada), 8 bits de datos (bit menos significativo primero; 1 = luz encendida), sin paridad, y un bit de parada (luz encendida). Cada byte ocupa 10 tiempos de bit. En reposo la línea queda en "1", por lo que el LED infrarrojo permanece ENCENDIDO mientras no se transmite.

Justificación: el UART resuelve en hardware la sincronización por carácter (bit de inicio), el muestreo a mitad de bit y la detección de errores de encuadre, sin un microcontrolador adicional ni temporización por software, que en Linux no es determinista. OOK es la modulación más simple de detectar con un comparador. El costo es que NRZ no garantiza transiciones dentro del byte; el bit de inicio de cada byte basta para resincronizar.

La velocidad (baudios) DEBE ser la misma en ambos Faros. El valor por defecto es 1200 baudios. Cualquier velocidad que soporte el UART PUEDE usarse; la velocidad máxima útil depende de la distancia, de la alineación, de la luz ambiente y del tiempo de respuesta del fototransistor (ver Apéndice B).

El enlace es semidúplex: los dos Faros comparten el mismo canal óptico y el receptor de un Faro puede ver el reflejo de su propio emisor (eco). La Sección 3.8 define cómo se maneja.

3.2. Dirección Física

Cada Faro tiene una dirección física de 16 bits.

- 0x0000 está reservada y NO DEBE asignarse.
- 0xFFFF es la dirección de difusión y NO DEBE asignarse a un Faro.
- Cualquier otro valor PUEDE usarse.

Un Faro DEBE generar su dirección de forma aleatoria y uniforme (con un generador criptográfico) entre 0x0001 y 0xFFFE, y DEBE guardarla en memoria no volátil para conservarla entre reinicios. La implementación de referencia la guarda en el archivo "~/.faronet_direccion" como 4 dígitos hexadecimales.

Un Faro DEBE poder rotar su dirección, es decir, reemplazarla por otra aleatoria nueva: a pedido del usuario, de forma periódica (OPCIONAL) y al activarse el botón de pánico. La rotación no requiere coordinación: el otro extremo aprende la dirección nueva (Sección 4.3).

Como las direcciones son aleatorias, dos Faros pueden elegir la misma con probabilidad aproximada de 1/65534 por par. Si ocurre, cualquiera de los dos PUEDE rotar su dirección.

3.3. Formato de la Trama

```
0                   1                   2                   3
0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
```

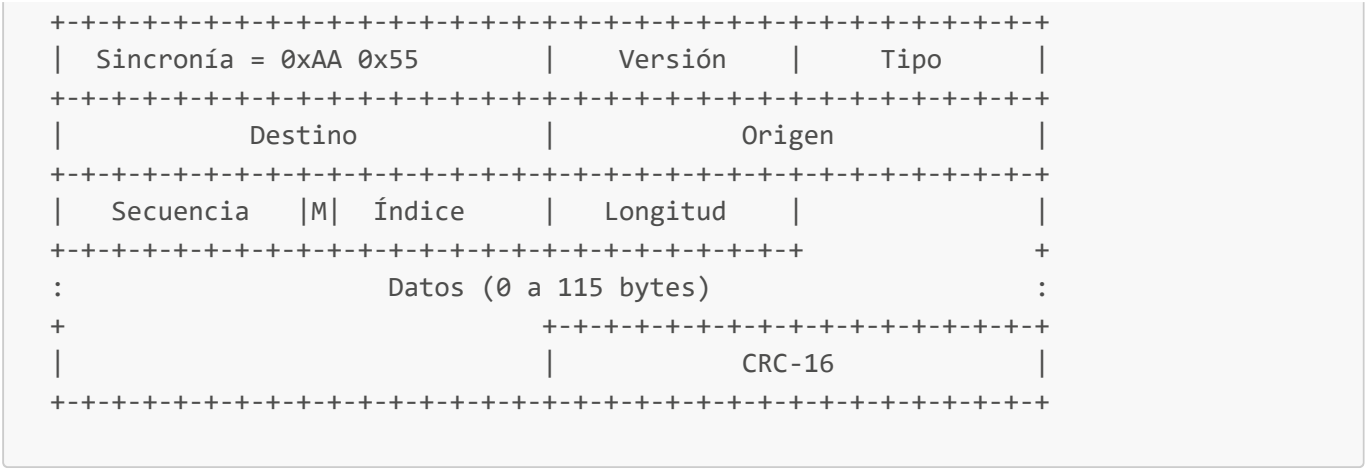


Figura 3: Trama de Capa 1

Tamaño: 2 (sincronía) + 9 (encabezado) + Longitud + 2 (CRC). El mínimo es 13 bytes (sin datos) y el máximo es 128 bytes.

3.4. Campos

- **Sincronía (2 bytes):** 0xAA seguido de 0x55. Marca el inicio de una trama. El patrón alterna bits (10101010 01010101), lo que ayuda al receptor a estabilizar el comparador.
- **Versión (1 byte):** versión del protocolo de Capa 1. Este documento define la versión 1.
- **Tipo (1 byte):** 1 = DATOS, 2 = ACK, 3 = EXTERNA (Sección 3.5).
- **Destino (2 bytes):** dirección física del Faro destinatario, o 0xFFFF para difusión.
- **Origen (2 bytes):** dirección física del Faro que transmite.
- **Secuencia (1 byte):** número de secuencia del emisor, módulo 256. Un emisor DEBE incrementarlo en cada trama nueva y DEBE repetirlo sin cambios en las retransmisiones de la misma trama.
- **M (1 bit, bit 7 del byte de Fragmento):** "vienen más"; vale 1 en todos los fragmentos de un mensaje excepto el último.
- **Índice (7 bits, bits 0 a 6 del byte de Fragmento):** posición del fragmento dentro del mensaje, empezando en 0.
- **Longitud (1 byte):** cantidad de bytes del campo Datos, de 0 a 115. Un valor mayor que 115 hace inválida la trama. Es el campo aparte que define el tamaño del único campo dinámico.
- **Datos (0 a 115 bytes):** carga útil.
- **CRC-16 (2 bytes):** código de verificación calculado sobre los bytes desde Versión hasta el final de Datos (Sincronía no se incluye).

3.5. Tipos de Trama

- **DATOS (1):** lleva un fragmento de un mensaje de una capa superior. Si el destino es una dirección individual, el receptor DEBE contestar con un ACK. Si el destino es 0xFFFF, NO DEBE contestar.
- **ACK (2):** confirma una trama DATOS o EXTERNA. Su campo Destino es el Origen de la trama confirmada, su campo Secuencia es la Secuencia de esa trama, y no lleva datos (Longitud = 0). Un ACK NO DEBE confirmarse.
- **EXTERNA (3):** igual que DATOS, pero los Datos son una trama creada fuera del Faro (por ejemplo con Scapy). El Faro la transporta sin interpretarla; el receptor la entrega a la aplicación como bytes crudos. Esto permite incluir protocolos ajenos sobre el Faro sin modificar la Capa 1.

Un receptor DEBE descartar en silencio las tramas de tipo desconocido.

3.6. Verificación de Integridad

El CRC-16 usa el polinomio x^16 + x^12 + x^5 + 1 (0x1021), valor inicial 0x0000, sin reflejar entrada ni salida y sin XOR final (variante conocida como CRC-16/XMODEM). Su valor de comprobación para la cadena ASCII "123456789" es 0x31C3. Se transmite en orden big-endian.

Este CRC detecta todos los errores de 1, 2 y 3 bits, todos los errores de cantidad impar de bits y todas las ráfagas de hasta 16 bits en tramas de hasta 128 bytes. NO es un mecanismo de autenticación (ver Sección 11.1).

3.7. Recepción y Resincronización

El receptor mantiene un buffer de bytes recibidos y aplica, en orden, estas reglas:

1. Busca la secuencia 0xAA 0x55. Descarta los bytes anteriores. Si el buffer termina en 0xAA, conserva ese byte (puede ser la mitad de una sincronía).
2. Si aún no hay 11 bytes (sincronía y encabezado), espera.
3. Si Versión no es una versión soportada o Longitud > 115, trata la sincronía como un falso arranque: descarta SOLO el primer byte del buffer y vuelve al paso 1.
4. Si aún no llegó la trama completa (13 + Longitud bytes), espera.
5. Si el CRC no coincide, descarta SOLO el primer byte y vuelve al paso 1.
6. Si todo es válido, entrega la trama y la quita del buffer.

Descartar un solo byte, y no todo el buffer, permite recuperar una trama válida que empezó dentro de bytes que parecían una trama.

Si pasa más de "timeout_byte" segundos sin recibir bytes y queda una trama a medias, el receptor DEBE abandonarla (descartando su primer byte y repitiendo el procedimiento) para no quedar bloqueado esperando bytes que se perdieron.

3.8. Acceso al Medio

El acceso al medio es semidúplex con confirmación por trama (parada y espera).

Maestro (envío con confirmación):

1. Asigna la siguiente Secuencia y transmite la trama.
2. Al terminar de transmitir espera "guarda_eco" segundos y descarta todo lo recibido mientras transmitía (su propio eco).
3. Espera un ACK con Origen = Destino de la trama y la misma Secuencia durante "espera_ack" segundos, que NO DEBE ser menor que "espera_ack_minima".
4. Si no llega, retransmite la MISMA trama (misma Secuencia). Tras "reintentos" intentos (3 por defecto) reporta un fallo.

Esclavo (recepción):

1. Ignora los ACK y las tramas cuyo Destino no es su dirección ni 0xFFFF.
2. Si el Destino es su dirección, espera "retardo_ack" segundos (para que el maestro termine de limpiar su eco) y contesta con un ACK.
3. Lleva, por cada Origen, la última Secuencia entregada. Si la trama repite esa Secuencia, es un duplicado (se perdió el ACK): la confirma otra vez pero NO DEBE entregarla de nuevo.

Los tiempos dependen de la velocidad. Con t_byte = 10 / baudios segundos (8N1), los valores RECOMENDADOS son:

Parámetro	Valor	a 1200 bd
t_byte	10 / baudios	8,33 ms
guarda_eco	max(0,01 s; 4 * t_byte)	33,3 ms
retardo_ack	2 * guarda_eco	66,7 ms
timeout_byte	max(0,5 s; 30 * t_byte)	500 ms
espera_ack_minima	retardo_ack + 13 * t_byte + timeout_byte	675 ms
timeout_fragmento	max(10 s; 256 * t_byte + 2 * espera_ack_minima)	10 s

Tabla 1: Tiempos de la Capa 1

"timeout_byte" es amplio porque el controlador del puerto serie entrega los bytes al programa en grupos, con pausas de varios tiempos de byte entre grupo y grupo. "guarda_eco" cubre los hasta ~32 bits que el controlador puede retener antes de entregarlos.

Después de transmitir, el emisor DEBE esperar al menos (bytes transmitidos * t_byte) segundos antes de considerar terminada la transmisión, aunque el controlador indique antes que ya la vació.

3.9. Fragmentación y Reensamblaje

Un mensaje de N bytes se divide en ceil(N/115) fragmentos de hasta 115 bytes (un mensaje vacío es un fragmento sin datos). El fragmento i lleva Índice = i y M = 1, salvo el último, que lleva M = 0. Como Índice tiene 7 bits, un mensaje admite como máximo 128 fragmentos, es decir 14720 bytes.

Cada fragmento se envía con confirmación (Sección 3.8). El mensaje solo se considera entregado si se confirmaron todos sus fragmentos.

El receptor reensambla por Origen:

- un fragmento con Índice = 0 inicia un mensaje nuevo de ese Origen (descartando el que estuviera a medias);
- un fragmento cuyo Índice no es el siguiente esperado hace que se abandone el mensaje a medias de ese Origen;
- el fragmento con M = 0 completa el mensaje;
- si entre fragmentos pasa más de "timeout_fragmento", el mensaje se abandona.

El Tipo del mensaje reensamblado es el del fragmento con Índice 0.

3.10. Difusión

Una trama con Destino 0xFFFF la reciben todos los Faros que la vean. No se confirma ni se retransmite. Un mensaje de varios fragmentos enviado por difusión se pierde completo si se pierde un solo fragmento. La difusión se usa para descubrir al otro Faro (Sección 4.3) y para medir el enlace en un solo sentido.

3.11. Manejo de Versiones

El byte Versión va en el encabezado de cada trama. Un receptor DEBE descartar las tramas cuya versión no soporta (regla 3 de la Sección 3.7). Una versión futura PUEDE cambiar cualquier campo posterior a Versión, pero DEBE conservar la sincronía 0xAA 0x55 y la posición del byte Versión, para que un receptor pueda identificarla. Un Faro que soporte varias versiones DEBE contestar en la misma versión de la trama recibida.

3.12. Interfaz DispositivoLuzAdaptador

La biblioteca DispositivoLuzAdaptador implementa esta sección sobre cualquier objeto "tipo puerto serie" (write, read, in_waiting, flush, reset_input_buffer). Ofrece:

- abrir(ruta, baudios, dirección): abre el UART en 8N1 y calcula los tiempos de la Tabla 1;
- enviar(trama) / recibir(timeout): nivel de trama, sin ACK;
- enviar_con_ack(...) / recibir_y_confirmar(...): papeles de maestro y esclavo;
- enviar_mensaje(destino, datos) / recibir_mensaje(timeout): mensajes de cualquier tamaño con fragmentación.

El nodo usa esta interfaz sin conocer el medio, de modo que el mismo código funciona con un puerto virtual (pruebas) o con el Faro.

3.13. Ejemplo

Trama DATOS de 0x566B a 0x2877, Secuencia 1, un solo fragmento, con los datos "Hola" (17 bytes):

```
AA 55 01 01 28 77 56 6B 01 00 04 48 6F 6C 61 DF 83
```

ACK correspondiente, de 0x2877 a 0x566B (13 bytes):

```
AA 55 01 02 56 6B 28 77 01 00 00 9D 7C
```

Intercambio de un mensaje de 2 fragmentos con pérdida de un ACK:

```
Maestro 566B                                  Esclavo 2877
   |--- DATOS seq=1 M=1 idx=0 (115 B) ------------->|
   |<------------------------------- ACK seq=1 ----|
   |--- DATOS seq=2 M=0 idx=1 (10 B) -------------->|
   |<------------------------------- ACK seq=2 --X |  (perdido)
   |      (vence espera_ack)                       |
   |--- DATOS seq=2 M=0 idx=1 (retransmisión) ----->|  duplicado:
   |<------------------------------- ACK seq=2 ----|  se confirma
   |                                               |  pero no se
   |                                               |  entrega dos
   |                                               |  veces
```

Figura 4: Retransmisión tras perder un ACK

4. El Faro como Enlace de la Mesh

Esta sección define cómo dos Faros transportan paquetes de la mesh (Sección 5) para que, ante el nodo, el otro Faro sea un vecino más, igual que una conexión TCP.

4.1. Encapsulado

Cada paquete de la mesh (512 bytes) se envía como un mensaje de Capa 1 de Tipo DATOS, es decir en ceil(512/115) = 5 fragmentos. El receptor solo entrega a la mesh mensajes de exactamente 512 bytes cuyo primer byte sea una versión de paquete soportada; los demás los descarta (por ejemplo, pruebas hechas con otras herramientas).

Con t_trama = (128 + 13) * t_byte + retardo_ack + guarda_eco (una trama máxima y su ACK), un paquete tarda t_paquete = 5 * t_trama. A 1200 baudios, t_paquete es de unos 6,4 s.

4.2. Acceso al Medio entre Faros

Antes de transmitir un paquete, un Faro:

1. escucha (atendiendo lo que llegue) durante retardo_ack más un tiempo aleatorio uniforme entre 0 y t_trama;
2. si están llegando bytes, espera t_trama y vuelve a revisar ("escuchar antes de hablar");
3. envía el paquete con confirmación por fragmento.

Si el envío falla, el Faro escucha durante un tiempo aleatorio U(1, 3) * (k + 1) * t_intento antes del intento k + 1, donde t_intento = 128 * t_byte + max(1 s; espera_ack_minima). Se hacen 3 intentos por paquete. La espera aleatoria evita que dos Faros que transmiten a la vez choquen de nuevo.

Cada Faro mantiene una cola de salida de como máximo 16 paquetes. Un anuncio de rutas nuevo reemplaza al anuncio que aún no salió, y los paquetes de datos y saludos se adelantan a los anuncios de rutas en espera. Si la cola está llena, los paquetes nuevos se descartan.

4.3. Descubrimiento del Otro Faro

Un Faro no necesita configurar la dirección del otro:

- mientras no la conoce, envía sus paquetes por difusión (0xFFFF, sin ACK);
- al recibir un paquete válido de la mesh, toma como dirección del otro Faro el Origen de esa trama, y desde entonces envía con ACK;
- si un paquete falla tras todos sus intentos, olvida la dirección y vuelve a la difusión.

Así, si el otro Faro rota su dirección, el enlace la aprende sola en el siguiente paquete. La dirección del otro Faro PUEDE fijarse manualmente; en ese caso no se olvida.

4.4. Vida del Enlace

Si durante "vida" segundos no llega ningún paquete del otro Faro, el enlace se declara caído y la mesh lo trata como una desconexión del vecino (Sección 5.5). Valores RECOMENDADOS: vida = max(60 s; 6 * t_paquete), y repetir el saludo cada max(5 s; 3 * t_paquete + 2 s). Con un enlace de 1200 baudios, el intervalo de anuncios de rutas DEBERÍA ser de al menos 20 s para no saturar la luz.

5. FaroNet: Red Mesh

5.1. Transporte

FaroNet funciona sobre cualquier enlace confiable y ordenado que transporte una secuencia de bytes: TCP sobre Ethernet o WiFi, o el enlace Faro de la Sección 4. Todos los paquetes miden 512 bytes, por lo que un receptor lee exactamente 512 bytes por paquete, sin delimitadores. Cada conexión es bidireccional y une a dos vecinos.

5.2. Formato del Paquete

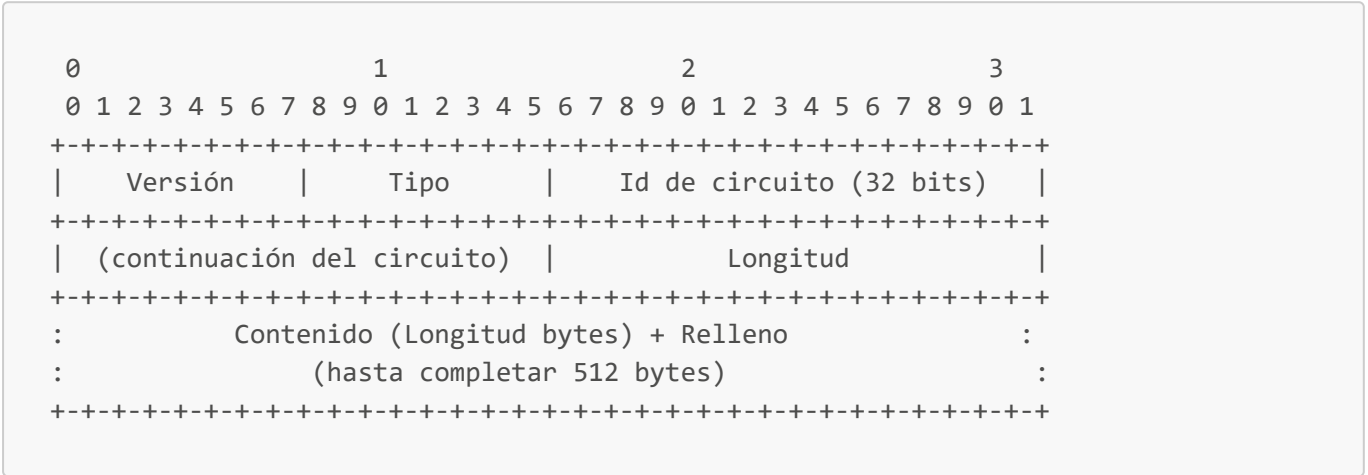


Figura 5: Paquete de FaroNet

- **Versión (1 byte):** 1 en este documento.
- **Tipo (1 byte):** ver la Sección 5.3.
- **Id de circuito (4 bytes):** reservado para circuitos virtuales negociados en una versión futura. En la versión 1 DEBE ser 0 al enviar y DEBE ignorarse al recibir.
- **Longitud (2 bytes):** bytes reales de Contenido, de 0 a 504.
- **Contenido y Relleno:** el Contenido ocupa los primeros Longitud bytes; el resto, hasta 504, es Relleno. El Relleno DEBE ser aleatorio (no ceros), para que el cifrado de capas inferiores o un observador no puedan distinguir contenido de relleno.

Un receptor DEBE descartar un paquete con Versión no soportada o Longitud mayor que 504.

5.3. Tipos de Paquete

Tipo	Nombre	Uso
1	HOLA	saludo entre vecinos
2	DATOS	mensaje directo enrutado
3	RUTAS	anuncio del vector de distancias
4	CEBOLLA	celda hacia el siguiente relay
5	MENSAJE	sellado del último relay al destino
10	DIR_PUBLICAR	nodo -> directorio: una ficha
11	DIR_CONSULTAR	nodo -> directorio: consulta

Tipo	Nombre	Uso
12	DIR_FICHA	directorio -> nodo: una ficha
13	DIR_FIN	directorio -> nodo: fin de fichas
14	DIR_RESPUESTA	directorio -> nodo: resultado

Tabla 2: Tipos de paquete

Un nodo DEBE ignorar en silencio los tipos que no conoce.

5.4. Saludo

- **Contenido de HOLA:** Bandera (1 byte) || Nombre (1 a 16 bytes).

Al abrir una conexión, cada extremo envía un HOLA con Bandera = 1 ("aún no te escucho, contéstame") y lo repite cada 5 s (con una variación aleatoria de +-30 %) hasta recibir el HOLA del vecino. Quien recibe un HOLA con Bandera = 1 DEBE contestar con un HOLA con Bandera = 0, que no se contesta (evita un ciclo). Un HOLA con Bandera = 1 recibido ya con la conexión establecida también DEBE contestarse, porque por luz el primer saludo pudo perderse.

Los paquetes recibidos antes del saludo se ignoran. Si el vecino usa el mismo nombre que el nodo, la conexión DEBE cerrarse.

5.5. Enrutamiento por Vector de Distancias

Cada nodo mantiene una tabla destino -> (distancia, siguiente salto, última actualización). La distancia se mide en saltos; 16 significa inalcanzable (como en RIP [RFC2453]).

Contenido de RUTAS: una o más entradas LongNombre (1 byte) || Nombre || Distancia (1 byte) La primera entrada es el propio nodo con distancia 0. Si las entradas no caben en 504 bytes, el anuncio se trunca.

- Al conectarse un vecino, su distancia es 1 y se anuncia la tabla a todos los vecinos.
- Al recibir un anuncio de V, para cada entrada (D, d): la distancia candidata es min(d + 1, 16). Si la ruta actual a D ya pasa por V, se acepta el nuevo valor aunque empeore; si no, solo se reemplaza si la candidata es menor (Bellman-Ford).
- Envenenamiento en reversa: al anunciar a V, las rutas cuyo siguiente salto es V se anuncian con distancia 16.
- Actualizaciones disparadas: todo cambio en la tabla se anuncia de inmediato, además del anuncio periódico (cada 5 s por defecto; 20 s o más sobre el Faro).
- Una ruta sin noticias durante 3 intervalos pasa a 16; en el siguiente ciclo se borra. Al desconectarse un vecino, todas las rutas que pasan por él pasan a 16 y se anuncian.

5.6. Paquetes DATOS

- **Contenido:** TTL (1) || LongDestino (1) || Destino || Mensaje

El paquete DATOS lleva el destino pero NO el origen. Cada nodo que lo reenvía DEBE decrementar el TTL (valor inicial 16) y DEBE descartarlo si llega a 1 sin haber llegado a destino, o si no tiene ruta. El Mensaje de un

paquete DATOS NO va cifrado de extremo a extremo: DATOS es para pruebas y diagnóstico; los mensajes privados DEBEN usar la cebolla (Sección 6).

5.7. Paquetes CEBOLLA y MENSAJE

- **Contenido:** TTL (1) || Destino (16, relleno con 0x00) || Carga

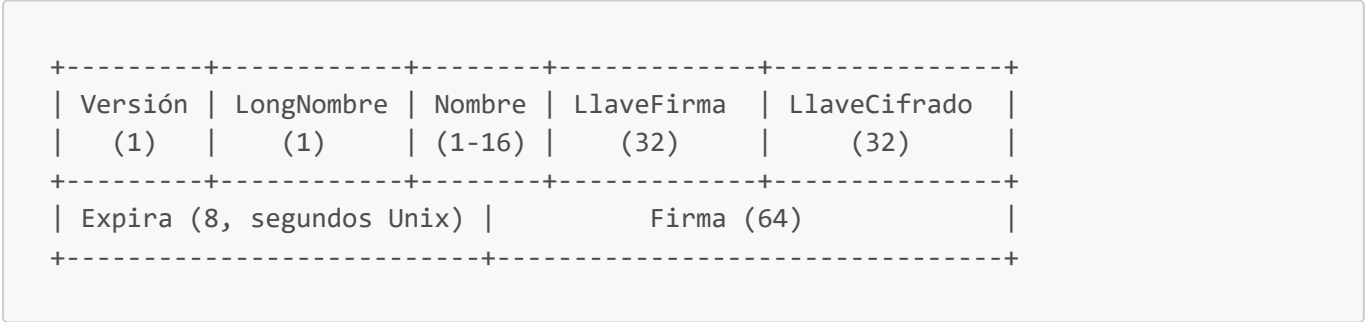
Se enrutan salto a salto igual que DATOS, pero el Destino mide siempre 16 bytes y la Carga tiene tamaño fijo por tipo (487 bytes en CEBOLLA, 278 en MENSAJE), de modo que el largo del contenido no filtra información. Cuando el Destino es el propio nodo, la Carga se entrega a la cebolla (Sección 6).

5.8. Identidad y Fichas

Cada nodo tiene dos pares de llaves:

- firma, Ed25519 [RFC8032], de larga vida: autentica las fichas del nodo; su semilla (32 bytes) se guarda en almacenamiento local con permisos solo para el dueño;
- cifrado, X25519 [RFC7748]: nunca se escribe a disco, se genera al arrancar y rota periódicamente (cada 3600 s por defecto o a pedido). Tras rotar, la llave anterior se acepta durante un periodo de gracia de 600 s y después se olvida.

Formato de una ficha (versión 1):



La Firma es Ed25519 sobre todos los campos anteriores. Una ficha vale 300 s desde su firma y el nodo la vuelve a publicar cada 60 s y en cuanto rota su llave de cifrado. Una ficha es válida si su firma es correcta, no ha vencido y su vencimiento no está a más de 3600 s en el futuro. La "huella" de un nodo, para compararla a simple vista, son los primeros 8 bytes (en hexadecimal) de BLAKE2b [RFC7693] de su LlaveFirma.

5.9. Directorio de Nodos

El directorio es un servicio TCP que usa el mismo paquete de 512 bytes. Cada operación abre una conexión nueva y espera la respuesta como máximo 5 s.

- **Publicar:** el nodo envía DIR_PUBLICAR con la ficha como contenido. El directorio contesta DIR_RESPUESTA con Código (1 byte: 0 = aceptada, 1 = rechazada) || Texto (motivo en UTF-8).
- **Consultar:** el nodo envía DIR_CONSULTAR con un nombre (o vacío para todos). El directorio envía un DIR_FICHA por ficha vigente y al final un DIR_FIN vacío.

Al publicar, el directorio DEBE rechazar una ficha:

- con firma inválida, vencida o con vencimiento demasiado lejano;

- cuyo nombre ya esté registrado, con otra LlaveFirma, en una ficha que no ha vencido (nadie puede apropiarse del nombre de un nodo activo);
- que venza antes que la ficha registrada para ese nombre (evita repetir fichas viejas capturadas).

Los nodos NO confían en el directorio: DEBEN verificar la firma de cada ficha que reciben y descartar las inválidas. El directorio guarda las fichas solo en memoria y DEBE borrarlas al vencer.

6. Enrutamiento por Capas (Cebolla)

6.1. Objetivo y Modelo

Ningún nodo intermedio debe conocer a la vez el origen y el destino de un mensaje privado. El emisor elige tres relays y envuelve el mensaje en una capa de cifrado por relay. Cada relay quita su capa y solo descubre el siguiente salto. El esquema es de una sola vía (sin circuito ni camino de regreso), como una versión simplificada de Sphinx [SPHINX]: cada celda es autocontenida.

```
Emisor ----> R1 ----> R2 ----> R3 ----> Destino
             ve:      ve:      ve:      ve:
             vecino   R1 y R3  destino  el mensaje,
             y R2              y un     sin origen
                               sellado  (salvo firmado)
```

R1 recibe la celda de un vecino de la mesh y el paquete no trae origen: no sabe si ese vecino es el emisor o solo la reenvió.

6.2. Selección de la Ruta

Antes de cada envío, el emisor DEBE consultar el directorio y verificar las fichas. Elige 3 relays distintos, al azar (con un generador criptográfico), entre los nodos del directorio para los que tiene ruta, excluyendo a sí mismo y al destino. Por lo tanto, la mesh necesita al menos 5 nodos con ficha válida. Si no hay 3 relays alcanzables, el envío DEBE fallar en lugar de usar menos saltos.

6.3. Derivación de Llaves

Para el relay i, el emisor genera una llave X25519 efímera e_i. Con la llave de cifrado pública del relay, P_i (de su ficha):

```
s_i   = HSalsa20(X25519(e_i, P_i))    (crypto_box_beforenm)
K     = BLAKE2b-512(Versión || s_i || pub(e_i) || P_i,
                    llave = "faronet-cebolla-v1")
kf_i  = K[0..31]     (llave de flujo)
km_i  = K[32..63]    (llave de MAC)
```

Flujo(k, n): los primeros n bytes del flujo de cifrado XSalsa20 [XSALSA20] con llave k y nonce 0^24, obtenidos como el texto cifrado (sin la etiqueta de 16 bytes) de crypto_secretbox aplicado a 0^n. Como cada llave se usa en un solo mensaje, el nonce fijo no se repite con la misma llave.

MAC(k, x) = BLAKE2b(x, tamaño = 8, llave = k).

La Versión entra en la derivación: alterarla hace fallar el MAC.

6.4. Formato de la Celda

La celda es la Carga de un paquete CEBOLLA y mide 487 bytes:

Versión (1)	Efímera (32)	MAC (8)	Bloque (446)

Sea S = 56 el tamaño de una cabecera de salto:

Siguiente (16, 0x00)	EfimeraSig (32)	MACsig (8)

El Bloque que ve el primer relay contiene, cifradas en capas, las 3 cabeceras y el sellado (Sección 6.7):

```
446 = 3 * 56 + 278
```

6.5. Procesamiento en el Relay

Al recibir una celda dirigida a él, un relay:

1. Descarta la celda si no mide 487 bytes o si su Versión no está soportada (sin reenviarla).
2. Para cada llave de cifrado propia aceptada (la vigente y las que están en gracia), deriva (kf, km) con la Efímera de la celda y compara MAC(km, Bloque) con el campo MAC en tiempo constante. Si ninguna coincide, descarta la celda.
3. Si la Efímera ya se vio en los últimos 600 s, descarta la celda (anti-repetición). La Efímera se recuerda solo después de verificar el MAC.
4. Calcula D = (Bloque || 0^56) XOR Flujo(kf, 502) y separa
   Cabecera = D[0..55] y Resto = D[56..501] (446 bytes).
5. Si EfimeraSig = 0^32, es el último relay: Siguiente es el nombre
   del destino. Envía un paquete MENSAJE a ese destino con
   Carga = Resto[0..277] (el sellado).
6. Si no, envía un paquete CEBOLLA a Siguiente con la celda Versión || EfimeraSig || MACsig || Resto.

El Bloque mantiene su tamaño en cada salto: al quitar una cabecera, el relay agrega 56 bytes al final ("relleno") que resultan de descifrar ceros con su propio flujo.

6.6. Construcción de la Celda en el Emisor

1. Deriva (kf_i, km_i) para los 3 relays y calcula
   F_i = Flujo(kf_i, 502).
2. Calcula el relleno determinista que el último relay verá al final de su Bloque (idea del "filler" de Sphinx):

```
r = vacío
para i = 1, 2:
  L = largo(r) + 56
  r = (r || 0^56) XOR (últimos L bytes de F_i)
```

El resultado mide 112 bytes. Como depende solo de las llaves, el MAC de cada salto lo cubre y ningún relay puede cambiarlo sin ser detectado.

3. Interior = NombreDestino(16) || 0^32 || 0^8 || Sellado(278)
   Bloque_3 = (Interior XOR F_3[0..333]) || r (446 bytes)
   MAC_3 = MAC(km_3, Bloque_3)

4. Para i = 2, 1:
   Cab = NombreRelay(i+1) || pub(e_(i+1)) || MAC_(i+1)
   Bloque_i = ((Cab || Bloque_(i+1)) XOR F_i)[0..445]
   MAC_i = MAC(km_i, Bloque_i)

5. Celda = Versión || pub(e_1) || MAC_1 || Bloque_1, que se envía al primer relay en un paquete CEBOLLA.

6.7. Sellado y Mensaje Plano

El sellado es crypto_box_seal [NACL] del mensaje plano hacia la llave de cifrado del destino. Mide 278 bytes: 48 de sobrecarga (llave efímera de 32 y etiqueta de 16) y 230 de mensaje plano.

Mensaje plano (230 bytes, siempre completo con ceros):

```
Banderas (1) || Largo (2) || Cuerpo (Largo bytes) || 0x00...
```

Banderas	Nombre	Cuerpo	Texto max.
0	anónimo	Texto	227 B
1	firmado	Remitente(16) || Box(Texto)	171 B
2	público	Tema(16) || Texto	211 B

Tabla 3: Variantes del mensaje plano

- **anónimo:** el destinatario no sabe quien lo envió y no puede contestar.
- **firmado:** Box(Texto) es crypto_box [NACL] (nonce de 24 bytes, etiqueta de 16) de la llave de cifrado del remitente a la del destinatario. El destinatario busca la ficha del Remitente en el directorio y abre la caja: si abre, el remitente queda autenticado. Si no abre, PUEDE refrescar el directorio una vez (no más de una vez cada 10 s) por si el remitente rotó su llave. El nombre del remitente viaja dentro del sellado: ningún relay lo ve. Para contestar, el destinatario envía otro mensaje firmado.
- **público:** va sellado hacia un nodo gateway. Tema son 1 a 16 bytes rellenos con 0x00. Nunca lleva remitente. Es el único tipo de mensaje que PUEDE salir de FaroNet (Sección 7).

El destinatario prueba sus llaves de cifrado aceptadas. Si ninguna abre el sellado, o el plano está mal formado, lo descarta. También descarta un sellado cuyos primeros 32 bytes (la efímera del sellado) ya vio en los últimos 600 s, porque un relay malicioso podría reentregarlo. Un nodo que no es gateway DEBE descartar los mensajes públicos.

6.8. Anti-repetición

Cada relay y cada destinatario recuerda en memoria, durante 600 s, las llaves efímeras ya procesadas (como máximo 4096). Si la memoria se llena de entradas vigentes, las celdas nuevas DEBEN descartarse (falla cerrada) en lugar de olvidar entradas vigentes.

6.9. Resumen de Tamaños

Elemento	Bytes
Paquete FaroNet	512
Contenido máximo	504
Encabezado enrutado (TTL + Destino)	17
Celda (Carga de CEBOLLA)	487
Encabezado de celda	41
Bloque	446
Cabecera de salto	56
Sellado (Carga de MENSAJE)	278
Mensaje plano	230

Tabla 4: Tamaños fijos

7. Gateway y Aplicación Pública en el Edge

7.1. Función del Gateway

El gateway es un nodo de la mesh con una llave de firma Ed25519 PROPIA, distinta de la de su ficha. Cuando la cebolla le entrega un mensaje público:

1. valida que el Tema cumpla ^[a-z0-9_-]{1,16}$ y que el texto, sin espacios a los extremos, mida de 1 a 300 caracteres; si no, lo descarta;
2. lo publica con POST /gateway/publicar (Sección 7.3);
3. si recibe 409 (el mismo mensaje en el mismo segundo parece una repetición), espera 1,1 s y reintenta una vez con otra marca de tiempo.

Además, cada 60 s (por defecto) consulta el directorio de la mesh y envía las fichas con POST /gateway/directorio, en grupos de 64. El gateway NO DEBE guardar a disco ni imprimir el texto de los mensajes.

7.2. Autenticación de las Peticiones del Gateway

Cada petición del gateway lleva dos encabezados HTTP [RFC9110]:

- **X-Faro-Ts:** marca de tiempo Unix en segundos (entero decimal).
- **X-Faro-Firma:** firma Ed25519 en hexadecimal (128 dígitos) sobre

```
METODO "\n" RUTA "\n" TS "\n" hex(SHA-256(cuerpo))
```

donde RUTA es la ruta de la URL (por ejemplo "/gateway/publicar") y hex() usa minúsculas [RFC6234].

El edge DEBE rechazar con 401 la petición si la marca de tiempo se aleja más de 60 s de su reloj, si la firma no tiene 64 bytes o si no verifica con la llave pública configurada (GATEWAY_PUBKEY). El edge DEBE recordar cada firma válida durante al menos el doble de la ventana (120 s) y rechazar con 409 una firma repetida. Como la RUTA está firmada, una firma no sirve para otra ruta.

El gateway DEBERÍA enviar un encabezado User-Agent propio (por ejemplo "faronet-gateway/1"): algunos proveedores de edge bloquean los agentes por defecto de bibliotecas HTTP.

7.3. Recursos HTTP del Edge

Método	Ruta	Quien	Respuestas
POST	/gateway/publicar {"tema","texto"}	gateway (firmado)	204, 400, 401, 409, 413, 429
POST	/gateway/directorio {"fichas":[hex,...]}	gateway (firmado)	200 {"aceptadas", "rechazadas"}
GET	/	cualquiera	200 HTML
GET	/sala/{tema}	navegador	200 HTML
GET	/sala/{tema}/ws	navegador	101 WebSocket
GET	/sala/{tema}/historial	cualquiera	200 JSON
GET	/sala/{tema}/feed.atom	cualquiera	200 Atom
GET	/directorio	cualquiera	200 JSON
GET	/directorio/{nombre}	cualquiera	200, 404
PUT	/buzon/{id} (1024 bytes)	cualquiera	201, 400, 409, 413, 429
GET	/buzon/{id}	cualquiera	200 (y borra), 404

Tabla 5: Recursos del edge

- **Salas:** una por tema, con su propio estado (Durable Object). Los mensajes se difunden en vivo por WebSocket [RFC6455] de solo lectura y viven 24 h (como máximo 200 por sala). Al vencer DEBEN borrarse, no solo ocultarse. El historial devuelve los últimos 50 mensajes vigentes.
- **Feed:** Atom [RFC4287] por tema, del más nuevo al más viejo, con el XML escapado y el autor siempre "anónimo".
- **Directorio público:** réplica de solo lectura. El edge DEBE verificar cada ficha por su cuenta con las mismas reglas de la Sección 5.9; el gateway puede omitir fichas, pero no inventarlas ni alterarlas.

Expone la ficha cruda para que cualquiera re-verifique la firma.

7.4. Buzón Muerto

Dos personas que comparten un secreto (acordado por un canal privado, por ejemplo un mensaje firmado de FaroNet) derivan:

```
id    = hex(BLAKE2b(secreto, tamaño = 16,
                    llave = "faronet-buzon-id"))
llave = BLAKE2b(secreto, tamaño = 32,
                llave = "faronet-buzon-llave")
```

El blob mide siempre 1024 bytes:

```
Nonce (24) || crypto_secretbox(llave,
                               Largo(2) || Mensaje || 0x00...)
```

con un Mensaje de hasta 982 bytes. El edge solo ve un id opaco (32 dígitos hexadecimales) y bytes cifrados de tamaño fijo. DEBE rechazar blobs de otro tamaño (400), DEBE rechazar un depósito sobre un id vigente (409), DEBE entregar y borrar el blob en una sola operación atómica, de modo que dos lectores nunca reciban el mismo blob, y DEBE borrarlo a las 24 h aunque nadie lo recoja. El secreto DEBERÍA tener al menos 16 caracteres aleatorios.

7.5. Límite de Peticiones

El edge limita las peticiones por cliente en ventanas fijas de 60 s: 120 lecturas, 30 usos del buzón y 120 intentos al gateway (los intentos con firma inválida también cuentan). Al exceder el límite contesta 429 con "Retry-After: 60". La clave del cliente es SHA-256(IP || día || sal) truncado a 16 bytes: cambia cada día, se guarda solo en memoria y nunca contiene la IP.

7.6. Política de No Registro

El edge NO DEBE persistir encabezados como CF-Connecting-IP o User-Agent, ni direcciones IP, en ningún almacenamiento. La observabilidad del proveedor (registros de peticiones) DEBE estar desactivada en la configuración de despliegue. Las respuestas llevan "Cache-Control: no-store", "Referrer-Policy: no-referrer", "X-Content-Type-Options: nosniff", no ponen cookies, y las páginas HTML llevan una Content-Security-Policy con un nonce distinto por petición (default-src 'none').

8. Manejo de Versiones

Todos los formatos llevan un campo de versión propio y todos los campos de tamaño variable tienen su largo en un campo aparte:

Formato	Campo de versión	Campos de largo
Trama de Capa 1	Versión (byte 2)	Longitud

Formato	Campo de versión	Campos de largo
Paquete FaroNet	Versión (byte 0)	Longitud
HOLA / DATOS / RUTAS	(del paquete)	LongDestino, LongNombre
Ficha	Versión (byte 0)	LongNombre
Celda de cebolla	Versión (byte 0, ligada al MAC)	(tamaño fijo)
Mensaje plano	Banderas	Largo
Firma del gateway	(ruta /gateway/)	Content-Length

Tabla 6: Versiones y largos

Reglas para versiones futuras:

- Un receptor DEBE descartar, sin procesarlo ni reenviarlo, un formato cuya versión no soporta.
- Una versión nueva DEBE conservar la posición del campo de versión de la anterior.
- Una implementación PUEDE soportar varias versiones a la vez (la cebolla ya mantiene un conjunto de versiones soportadas) y DEBE contestar en la versión que recibió.
- Cambiar el número de saltos de la cebolla (3) cambia todos los tamaños de la Tabla 4 y por lo tanto REQUIERE una versión nueva de la celda.
- Una versión nueva de la API del edge DEBERÍA publicarse bajo un prefijo de ruta nuevo (por ejemplo "/v2/gateway/"); como la ruta está firmada, las firmas de una versión no sirven en otra.

9. Consideraciones de IANA

Este documento no requiere acciones de IANA. FaroNet no usa puertos asignados: los puertos TCP de nodos y directorio se configuran por línea de comandos (la implementación de referencia usa 9000 para el directorio en sus ejemplos). Los encabezados X-Faro-Ts y X-Faro-Firma son privados de esta aplicación.

10. Consideraciones de Privacidad

Esta sección sigue la estructura de [RFC6973].

10.1. Modelo de Amenazas

Objetivo: que ningún adversario considerado pueda vincular a un usuario con el contenido y el destinatario de sus mensajes privados.

Adversario	Qué puede ver	Qué NO puede ver
Observador del medio físico (ve la luz)	Cuando transmite cada Faro; direcciones físicas, nombres en HOLA y RUTAS (en claro); el tamaño fijo de los paquetes	Contenido de celdas y sellados; origen y destino finales de un mensaje privado
Nodo malicioso de la mesh (relay)	Su vecino anterior y el siguiente salto; si es el último relay, el destino	Origen y destino a la vez; el contenido; si el vecino anterior es el emisor

Adversario	Qué puede ver	Qué NO puede ver
Destinatario	El texto; el remitente solo si el mensaje es firmado	El origen de un mensaje anónimo
Operador del gateway	Tema y texto de los mensajes públicos (son públicos); fichas del directorio	Quién originó el mensaje; los mensajes privados
Proveedor del edge (Cloudflare)	IP del gateway y de los lectores; mensajes públicos; ids opacos y blobs cifrados del buzón; horarios	Autores dentro de FaroNet; contenido del buzón; mensajes privados
Directorio	Nombres, llaves públicas y la conexión de quien publica o consulta	El tráfico de la mesh
Acceso físico al Faro	Lo que haya en disco y memoria si llega antes del botón de pánico	Llaves borradas por el botón; tráfico cifrado con llaves ya olvidadas

Tabla 7: Adversarios considerados

Fuera del alcance: un observador global capaz de ver todos los enlaces a la vez, y la colusión de los tres relays de una ruta.

10.2. Vigilancia

Un observador de un enlace ve paquetes de 512 bytes con relleno aleatorio, y celdas y sellados de tamaño fijo: no puede inferir la longitud del mensaje ni distinguir un mensaje anónimo de uno firmado. No hay mezcla ni retardos deliberados: un observador que vea varios enlaces puede correlacionar por tiempos (Sección 10.4).

En el Faro, la Capa 1 NO cifra: las direcciones físicas, los anuncios de rutas (nombres de nodos) y los saludos viajan en claro por la luz. Solo el contenido de CEBOLLA y MENSAJE va cifrado. Los paquetes DATOS (Sección 5.6) tampoco van cifrados y NO DEBEN usarse para contenido privado.

10.3. Almacenamiento y Retención

- Nodos y gateway: lo único que se escribe a disco es la semilla de la llave de firma y la dirección física. Las llaves de cifrado, tablas de rutas, fichas y efímeras vistas viven solo en memoria.
- Directorio: fichas solo en memoria, borradas al vencer (300 s).
- Edge: mensajes de salas y blobs del buzón con TTL de 24 h, borrados por una alarma; fichas hasta su vencimiento; contadores de límite y firmas vistas solo en memoria.
- Ningún componente registra direcciones IP, direcciones físicas ni identificadores de usuario; la salida por terminal muestra solo nombres de nodos.

10.4. Correlación

- Las direcciones físicas permiten seguir un Faro en el tiempo; por eso son aleatorias y rotables (Sección 3.2). Entre rotaciones, las tramas de un mismo Faro son vinculables.
- Los nombres de nodo son seudónimos estables mientras no se cambien; la llave de firma no rota.
- Un adversario que observe la entrada y la salida de la mesh puede correlacionar por tiempos, porque no hay mezcla.

- La clave de límite del edge cambia cada día, para que no sirva como identificador de largo plazo.

10.5. Identificación

Un mensaje anónimo no lleva ningún identificador del emisor. Un mensaje firmado identifica al remitente solo ante el destinatario. Un mensaje público nunca lleva remitente. El gateway publica sin autor ("anónimo").

10.6. Uso Secundario y Divulgación

El edge publica solo lo que el usuario marcó como público. Los mensajes privados nunca llegan al gateway ni al edge. El proveedor del edge ve las conexiones (IP, horarios); para ocultar su IP, un usuario del buzón DEBERÍA hacer sus peticiones desde un nodo detrás de la mesh o desde otra red.

10.7. Exclusión y Participación del Usuario

El usuario decide, mensaje por mensaje, entre anónimo, firmado y público. El botón de pánico le permite borrar su estado sensible y cambiar de dirección física en cualquier momento: el Faro sobrescribe con ceros y borra los archivos de llaves, olvida las llaves en memoria, rota la dirección física y lo indica con los LEDs. Si el Faro es el gateway, además detiene el proceso del gateway y borra su llave de firma, que es estado sensible: tras el pánico el gateway no puede publicar hasta que su operador reinstale la llave (o genere una nueva y actualice GATEWAY_PUBKEY en el edge).

10.8. Mitigaciones Aplicadas

- Minimización de datos: paquetes sin origen, tamaños fijos, relleno aleatorio, sin registros, TTL en todo lo almacenado.
- Anonimato: cebolla de 3 saltos con relays elegidos al azar.
- Seudonimato: nombres de nodo y direcciones físicas rotables.
- Confidencialidad: cifrado extremo a extremo con PyNaCl/libsodium, sin primitivas criptográficas propias.
- Secreto hacia adelante parcial: la llave de cifrado rota y la anterior se olvida tras 600 s.

11. Consideraciones de Seguridad

11.1. Capa 1

- El CRC-16 detecta errores de transmisión pero no autentica: un atacante con un emisor infrarrojo puede inyectar tramas válidas, falsificar un Origen o enviar ACK falsos. La integridad y la autenticidad del contenido las dan las capas superiores (MAC por salto, sellado, firmas).
- La luz puede interferirse (deslumbramiento, luz ambiente fuerte) y bloquearse; esto es una denegación de servicio que el diseño no evita. Los reintentos y la vuelta a la difusión solo recuperan fallas transitorias.
- Un atacante puede responder a la difusión para que un Faro "aprenda" una dirección falsa; el efecto es una denegación de servicio hasta el siguiente fallo, no una fuga de datos, porque las celdas siguen cifradas.

11.2. Mesh y Directorio

- Sybil: un atacante puede registrar muchos nodos y aumentar la probabilidad de controlar los tres relays de una ruta. El directorio no limita los registros.

- El directorio asigna nombres al primero que los registra (mientras su ficha no venza); un nodo que se apaga más de 300 s puede perder su nombre.
- Un nodo malicioso puede anunciar rutas falsas (distancia 1 a todo) para atraer tráfico, o descartar paquetes. La cebolla impide que lea o altere las celdas, pero no que las descarte.
- El TTL (16) y la distancia infinita (16) acotan los ciclos de enrutamiento.

11.3. Cebolla

- Los MAC por salto son de 64 bits, suficientes para impedir que un relay marque o altere celdas, pero no para otros usos.
- Secreto hacia adelante parcial: dentro de la vida de una llave de cifrado (y su gracia) el tráfico capturado puede descifrarse si se roba la llave. No hay un intercambio por mensaje.
- La llave de firma no rota: robarla permite suplantar al nodo (publicar fichas, firmar mensajes), no descifrar tráfico pasado.
- Anti-repetición por 600 s: una celda repetida después de ese tiempo se descifraría, pero para entonces la llave del relay puede haber rotado.
- En Python no se puede garantizar el borrado de la memoria; se sueltan las referencias. En una microSD, sobrescribir un archivo no garantiza borrar el bloque físico; guardar las llaves en RAM (por ejemplo /dev/shm) evita ese problema.

11.4. Gateway y Edge

- La llave del gateway es la única credencial del edge: si se roba, el atacante puede publicar en las salas y alterar la réplica del directorio (solo con fichas válidamente firmadas). DEBE guardarse con permisos solo para el dueño y fuera del repositorio; solo la llave pública va en la configuración del despliegue. El botón de pánico del Faro gateway la borra (Sección 10.7). Si pudo haber sido copiada, DEBE generarse una nueva y actualizarse GATEWAY_PUBKEY antes de volver a publicar.
- La ventana de 60 s exige que el reloj del gateway esté sincronizado (por ejemplo con NTP).
- El registro de firmas vistas y los contadores de límite viven en memoria: si la instancia se reinicia, se pierden. Una petición capturada podría repetirse en ese caso solo dentro de su ventana de 60 s.
- El límite por cliente usa la IP: varios usuarios detrás de la misma IP comparten el límite.
- Las páginas HTML usan una CSP estricta con nonce por petición para impedir la inyección de scripts mediante el texto de los mensajes, que además se escapa.

12. Referencias

12.1. Referencias Normativas

- **[RFC2119]** Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, marzo 1997.
- **[RFC8174]** Leiba, B., "Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words", BCP 14, RFC 8174, mayo 2017.
- **[RFC7748]** Langley, A., Hamburg, M. y S. Turner, "Elliptic Curves for Security", RFC 7748, enero 2016.
- **[RFC8032]** Josefsson, S. e I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)", RFC 8032, enero 2017.
- **[RFC7693]** Saarinen, M-J. y J-P. Aumasson, "The BLAKE2 Cryptographic Hash and Message Authentication Code (MAC)", RFC 7693, noviembre 2015.

- **[RFC6234]** Eastlake 3rd, D. y T. Hansen, "US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)", RFC 6234, mayo 2011.
- **[RFC9110]** Fielding, R., Ed., Nottingham, M., Ed. y J. Reschke, Ed., "HTTP Semantics", STD 97, RFC 9110, junio 2022.
- **[NACL]** Bernstein, D. J., Lange, T. y P. Schwabe, "The security impact of a new cryptographic library", LATINCRYPT 2012. Implementado por libsodium y PyNaCl.
- **[XSALSA20]** Bernstein, D. J., "Extending the Salsa20 nonce", 2011.

12.2. Referencias Informativas

- **[RFC2453]** Malkin, G., "RIP Version 2", STD 56, RFC 2453, noviembre 1998.
- **[RFC4287]** Nottingham, M., Ed. y R. Sayre, Ed., "The Atom Syndication Format", RFC 4287, diciembre 2005.
- **[RFC6455]** Fette, I. y A. Melnikov, "The WebSocket Protocol", RFC 6455, diciembre 2011.
- **[RFC6973]** Cooper, A., Tschofenig, H., Aboba, B., Peterson, J., Morris, J., Hansen, M. y R. Smith, "Privacy Considerations for Internet Protocols", RFC 6973, julio 2013.
- **[RFC7322]** Flanagan, H. y S. Ginoza, "RFC Style Guide", RFC 7322, septiembre 2014.
- **[SPHINX]** Danezis, G. e I. Goldberg, "Sphinx: A Compact and Provably Secure Mix Format", IEEE Symposium on Security and Privacy, 2009.

Apéndice A. Vectores de Prueba

A.1. CRC-16

CRC-16/XMODEM("123456789") = 0x31C3

A.2. Tramas

DATOS, Destino 0x2877, Origen 0x566B, Secuencia 1, Fragmento 0x00, Datos "Hola":

```
AA 55 | 01 | 01 | 28 77 | 56 6B | 01 | 00 | 04 | 48 6F 6C 61
| DF 83
```

ACK de esa trama (Destino 0x566B, Origen 0x2877, Secuencia 1):

```
AA 55 | 01 | 02 | 56 6B | 28 77 | 01 | 00 | 00 | 9D 7C
```

Una trama con 115 bytes de datos mide exactamente 128 bytes.

Apéndice B. Mediciones de la Implementación de Referencia

Mediciones entre dos Raspberry Pi Model B+ (ARMv6, 700 MHz) con Faros soldados. Las tablas de BER y tasa efectiva corresponden a 100 cm, en un cuarto a oscuras, sin otras fuentes de luz que pudieran afectar la comunicación. Las demás distancias se midieron en interior con luz ambiente normal. El detalle completo está en la documentación del proyecto.

BER (bytes sin trama):

Baudios	Faro A -> Faro B	Faro B -> Faro A
1200	0 / 8000 bits	0 / 8000 bits
1500	0 / 8000 bits	0 / 8000 bits
1800	0 / 40000 (< 7,5e-5)	0 / 40000 (< 7,5e-5)
2400	0 / 16000 bits	74 / 16000 (4,6e-3)

Tasa efectiva con el protocolo completo (1000 bytes, 9 tramas, ACK por trama), igual en ambos sentidos:

Baudios	Tiempo	Carga útil	Retransmisiones
1200	11,23 s	712 bit/s	0
1500	9,00 s	889 bit/s	0
1800	7,51 s	1065 bit/s	0
2400	-	-	falla: se pierden ACK

La eficiencia es del 59 % de la velocidad del enlace. La velocidad máxima confiable a 100 cm fue 1800 baudios.

El umbral del receptor se ajustó para 100 cm, 65 cm y 30 cm; las mediciones de 15 cm y 7,5 cm usan el ajuste de 30 cm.

Distancia	Velocidad máxima confiable	Carga útil con ACK (ambos sentidos)
7,5 cm	3000 baudios	~1762 bit/s
15 cm	3600 baudios	~2105 bit/s
30 cm	3600 baudios	~2105 bit/s
65 cm	2700 baudios	~1590 bit/s
100 cm	1800 baudios	~1065 bit/s

En todas las distancias el BER a la velocidad máxima fue 0 en 40000 bits por sentido (< 7,5e-5 al 95 %). A 15 cm y 30 cm el máximo es el mismo: de cerca, el límite ya no es la cantidad de luz sino la velocidad de respuesta del fototransistor y del comparador. A 7,5 cm el límite es el exceso de luz: a 3600 baudios el BER sube a 1,7e-2 - 4,6e-2.

El umbral del comparador DEBERÍA ajustarse según la distancia.

Con dos grupos de nodos unidos solo por los Faros a 1800 baudios, un paquete de 512 bytes cruza la luz en unos 4,2 s; un mensaje directo de 5 saltos tardó 4,8 s y uno anónimo de 3 relays, entre 9,3 s y 21,7 s, según cuántas veces la ruta cruza la luz. Sobre WiFi, un mensaje anónimo tardó 0,31 s. En la Pi B+, firmar con Ed25519 cuesta 2,1 ms y abrir un sellado, 3,5 ms.

Apéndice C. Correspondencia con la Implementación

Sección	Archivo
3 Capa 1	faronet/faro/capa1.py
3.1, 10.7 LEDs y botón	faronet/faro/hardware.py
4 Faro como enlace	faronet/faro/enlace_faro.py
5.2, 5.3 Paquete	faronet/mesh/paquete.py
5.4 a 5.7 Nodo y rutas	faronet/mesh/nodo.py
5.8 Identidad y fichas	faronet/mesh/identidad.py
5.9 Directorio	faronet/mesh/directorio.py
6 Cebolla	faronet/mesh/cebolla.py
7.1, 7.2 Gateway	faronet/herramientas/ gateway.py, gateway_nodo.py
7.2 a 7.6 Edge	edge/src/*.ts
7.4 Cliente del buzón	faronet/herramientas/buzon.py

Direcciones de los Autores

Joshua Solís Fuentes
Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación
Cartago, Costa Rica

Jesús Andrey Solís Mora
Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computación
Cartago, Costa Rica